Topics:

- Python Resources
- Good Coding Practices
- Interpreting Error Messages
- Jupyter Notebook Secrets
- Writing Functions
- Taylor Series
- Search Engines and Generative AI
- Plotting

Python Resources

Here are some resources you can use to help you learn Python. Depending on the type of help you are looking for, some resources may be more helpful than others.

Python Education Subreddit: Community, general advice, and a place you can ask/browse questions about using Python.

W3Schools: Tutorials on Python programming from a low-level perspective (data structures, syntax, etc).

GeeksforGeeks: Tutorials that are more focused on concepts and examples of how to implement different tools. Highly recommended!

Programiz: More tutorials :)

Stack Overflow: One of the best debugging resources. A link isn't posted here, because the best way to use it is to Google/Bing an error message or a description of what you're stuck on and add the words "stack overflow" to your search.

Kaggle Lessons: Python walkthrough/crash course that you can work through at your own pace.

Python Tutor: Copy and paste your code in here to see a visualization of how the computer runs your code -- this is a great debugging tool.

Official Python Documentation: Go here to look under the hood of the Python programming language. There are also several links to more resources and tutorials here.

There are also docs for specific libraries, like **numpy** and **matplotlib.pyplot**

Good Coding Practices

What are good coding practices? These tips have come from class discussion in years past.

Strategize your problem-solving

- Plan out your problem-solving approach
- Write your code in separate steps, and review these steps to get a better understanding
- chunk your code into different small sections

Give your code style and readability

- Write comments to explain your code
- Name your variables sensibly, not just a, b, c
- Make your code readable to others
- Use a case when naming variables (e.g., `snake_case` or `camelCase`) or some type of consistent style

Make your code efficient

- Make parts of your code more reusable by writing functions for repeated calculations

Debug your code

- Run your code many times as you work, so you can spot where bugs are located
- Test your code as you go using print statements to see if the outputs are what you expect

Keep track of relevant information

- comment physical units into your code wherever physical quantities are represented
- Cite any sources in your notebook. One option is to use comments at the top/bottom of your code chunk

Interpreting Error Messages

When you get an error in Python, there are few different parts to watch out for:

- The **location** of the error is specified at the **very top** of the error message.
- The **explanation** of the error is given at the **very bottom** of the error message.
- The information in the middle of the error message can be useful, but usually has a lot of extraneous information. It's best to avoid reading through all of that stuff at first, and try to solve your error using what you know about the location of the error in your code and the explanation given at the very bottom of the message.
- A good strategy can be to copy the computer-worded explanation at the very bottom and paste it into a search engine to see how other people have solved that error before.

An example error message:

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[16], line 6
      3 L = np.zeros(50)
      4 i = 0
----> 6 for t in np.linspace(0, 5, 0.1):
      7     L[i] = t
      8     i += 1

File ~/anaconda3/lib/python3.12/site-packages/numpy/core/function_base.py:122, in linspa
     24 @array_function_dispatch(_linspace_dispatcher)
     25 def linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None,
     26             axis=0):
     27     """
     28     Return evenly spaced numbers over a specified interval.
     29
   (...)
    120
    121     """
--> 122     num = operator.index(num)
    123     if num < 0:
    124         raise ValueError("Number of samples, %s, must be non-negative." % num)

TypeError: 'float' object cannot be interpreted as an integer
```

In this example, the error message tells us that the **location** of the error is in line 6, where the loop is declared.

We can also see an **explanation** at the very bottom of the message, which tells us "'float' object cannot be interpreted as an integer."

A **float** in Python means a decimal. The only decimal in line 6 is 0.1, which is being used as the third argument in `np.linspace`. The third input to `np.linspace` is supposed to be an integer indicating the number of elements in the array. A float, like 0.1, will not work here.

In this example, a solution could be to switch to `np.arange` instead, where the third input is supposed to be the step size between elements in the array. Alternatively, 0.1 could be changed to an integer, like 50. Ultimately, it's up to whoever is fixing the code to determine which implementation makes more sense for their program.

Jupyter Notebook Secrets

There are a few tricks you can use within Jupyter notebooks that could solve an issue you've never seen before, or just make your life a little easier.

Quick access to documentation using `?`

Running this code in a Jupyter notebook shows the documentation for the `np.arange` function. This can be used for any function built into Python or imported from a library.

```
import numpy as np

np.arange?
```

```
Docstring:
arange([start,] stop[, step,], dtype=None, *, like=None)

Return evenly spaced values within a given interval.

``arange`` can be called with a varying number of positional arguments:

* ``arange(stop)``: Values are generated within the half-open interval
  ``[0, stop)`` (in other words, the interval including `start` but
  excluding `stop`).
* ``arange(start, stop)``: Values are generated within the half-open
  interval ``[start, stop)``.
* ``arange(start, stop, step)`` Values are generated within the half-open
  interval ``[start, stop)``, with spacing between values given by
  ``step``.

For integer arguments the function is roughly equivalent to the Python
built-in :py:class:`range`, but returns an ndarray rather than a ``range``
```

Running console code using `%`

If you ever need to run console code, use the percent sign. The most common example is if you don't have a library or module installed on your computer, and you try to import the library into your Jupyter notebook. Your computer will spit out an error reading something like, `ModuleNotFoundError: No module named 'numpy'`.

In that case, you can run the code below to install NumPy, or whatever module/library you are missing.

```
%pip install numpy
```

You might need to use pip3 instead, depending on the version of Python you are using on your computer.

```
%pip3 install numpy
```

Writing Functions

When writing a new function, it's good practice to write details about the function's purpose, inputs, and outputs. This is for your own reference so that you are able to more easily write and use your function, but also for others who may be collaborating with you. This includes peers and instructors, who can more easily understand what you are trying to do with your code when there are comments describing it.

You can use the template below to include appropriate commentary and define a function.

```
# function_name (input_name1, input_name2, input_name3)
# purpose of function
#
# input_name1: purpose, type
# input_name2: purpose, type
```

```
# input_name3: purpose, type
#
# output: output_name
# output purpose, type

def function_name(input_name1, input_name2, input_name3):
    body of function_name
    ...
    return output_name
```

Taylor Series

Here is a helpful [conceptual video on Taylor Series](#).

Search Engines and Generative AI

Some tips class discussions in years past are included below.

For using search engines like Google, and AI chatbots like ChatGPT:

- Use unique keywords when framing your query
- Likely someone has asked your question before on Google -- if there are only a small number of results you could try rewording it
- Google keywords alone for more general information
- When using ChatGPT, be more specific, let it know whether it answered your question, and answer it's follow-up questions
- With ChatGPT, you can copy your whole code into it and ask it to explain what each part does
- Ask questions about a specific method -- for example, search for "numpy array functions" instead of just "lists"
- Be aware, you can tell ChatGPT that it is wrong, but it still might not correct itself properly
- Sometimes ChatGPT can produce unnecessarily complex code even for a simple query

Plotting

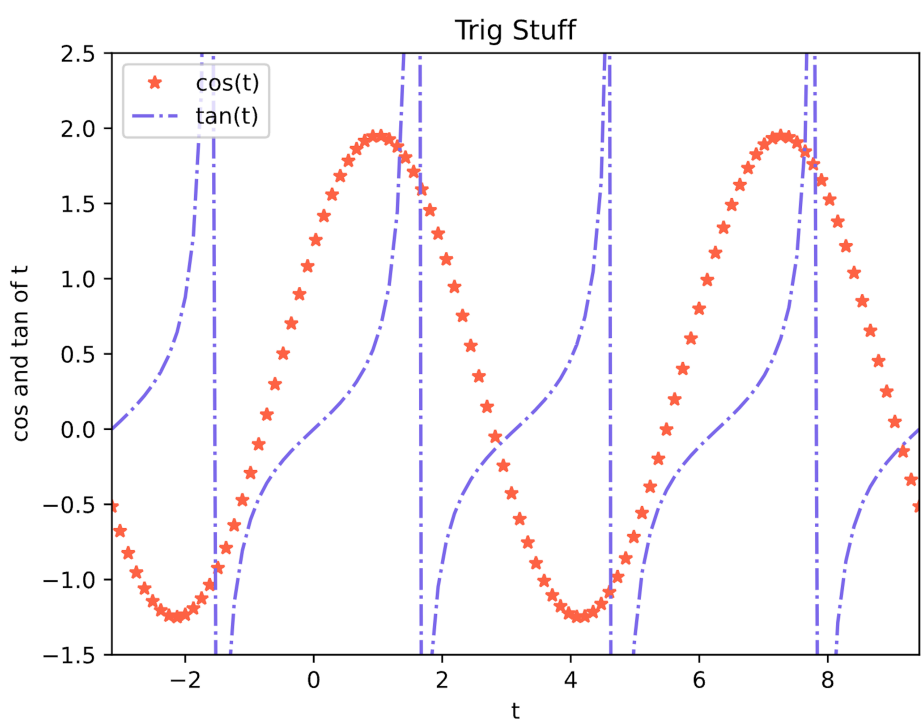
Here are a few of the key pieces of plotting code offered by Matplotlib:

Function/Attribute	Purpose	How to Use	Syntax
`import` statement	gain access to `plt` functions and attributes	Use it on it's own line, and run it once before doing any plotting. It's best to have it near the top of your notebook with other import statements so you can run it once and forget about it.	`import matplotlib.pyplot as plt`

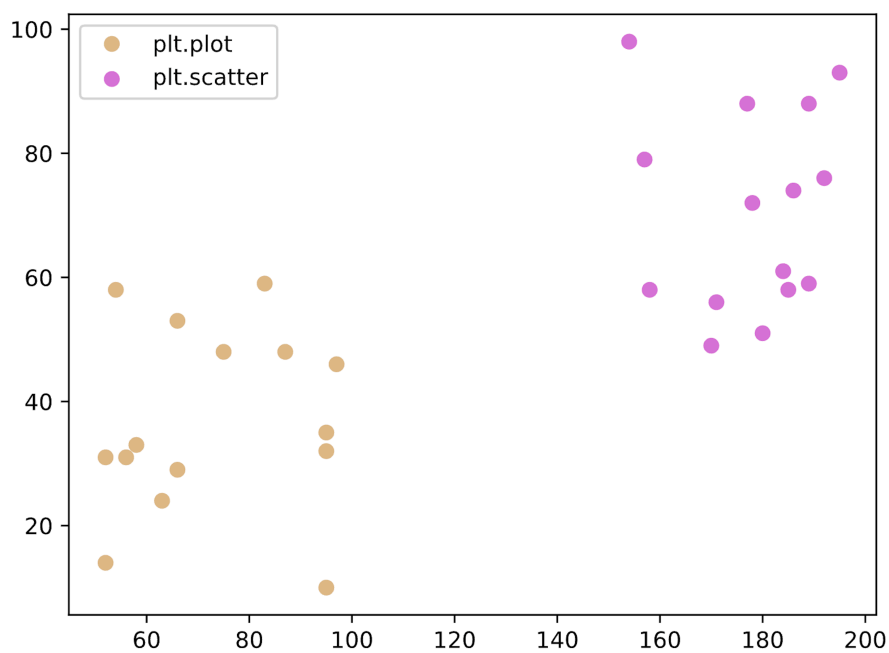
Function/Attribute	Purpose	How to Use	Syntax
`plot`	creation of axes and plotted values	Use it on it's own line and provide `x` and `y` inputs, plus other optional attributes.	`plt.plot(x, y, ...)`
`legend`	adding a legend to a plot	Use it on it's own line with parentheses, but no inputs required. Previous uses of `plot` should have labels (see below). The `loc` input is optional, and allows you to specify where the legend is placed.	`plt.legend(loc="center left")`
`label`	create a label to appear in the legend	Use it as an input to `plot`, and don't forget to use `legend` later on.	`plt.plot(x, y, ..., label="mylabel")`
`color`	specify the color of the plot	Use it inside `plot` as an optional input. Options are the names of colors, found here.	`plt.plot(x, y, ..., color="red")`
`marker`	specify the shape of each data point on the plot	Use it inside `plot` as an optional input. Options can be found here.	`plt.plot(x, y, ..., marker=".")`
`linestyle`	specify type of line connecting each data point	Use it inside `plot` as an optional input. Options are "None" for no line, or several line options found here.	`plt.plot(x, y, ..., linestyle="None")`
`xlim` and `ylim`	specify the bounds of the plot window	Use it on it's own line with two inputs: lower bound and upper bound	`plt.xlim(-50, 2000)`
`xlabel` and `ylabel`	create axis labels on the plot	Use it on it's own line with a "string" input indicating the desired axis label.	`plt.xlabel("Time (seconds)")`
`title`	create a title for your plot	Use it on it's own line with a "string" input indicating the desired title.	`plt.title("My Gorgeous Plot")`
`show`	ensure the plot displays in the output, sometimes not needed but always a good idea	Use it on it's own line with empty parentheses.	`plt.show()`

Below, you can see an example of these functionalities in use.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-np.pi, 3*np.pi, 100)

cost = 1.6 * np.cos(t-1) + 0.35
tant = 0.4 * np.tan(t)

# you can use plt.plot multiple times to create multiple plots on the same set of
axes
# the optional attributes can be left out and/or input in whatever order you please
plt.plot(t, cost, color="tomato", linestyle="None", marker="*", label="cos(t)")
plt.plot(t, tant, linestyle="dashdot", label="tan(t)", color="mediumslateblue")

# the functions below all modify the axes and plots created above
plt.legend(loc="upper left")
plt.xlim(-np.pi, 3*np.pi)
plt.ylim(-1.5, 2.5)
plt.xlabel("t")
plt.ylabel("cos and tan of t")
plt.title("Trig Stuff")

# this makes the plot show up!
plt.show()
```



Another useful function is `plt.scatter`. This creates a scatter plot similar to when you set `linestyle="None"` in a regular `plt.plot` function call. See below for an example.

Additional note: when using `linestyle="None"` with `plt.plot`, you must also specify a `marker` shape. Without this, you might not see anything at all when you create your plot.

```
# we can use np.random to generate some numbers for plotting
x1 = np.random.randint(50, 100, 15)
y1 = np.random.randint(10, 60, 15)

x2 = np.random.randint(150, 200, 15)
y2 = np.random.randint(40, 100, 15)

# creating a scatter plot with plt.plot
plt.plot(x1, y1, linestyle="None", marker="o", color="burlywood", label="plt.plot")

# creating a scatter plot with plt.scatter -- by default the points are shaped like
marker="o"
plt.scatter(x2, y2, color="orchid", label="plt.scatter")

plt.legend()
plt.show()
```